Q1. (a) - From statistic 2B, the total grades for the 4 male students is $4 \times 74.5 = 298$. - From statistic 4A, the total grades for the 3 undergrad male students is $3 \times 70 = 210$. - Out of the 4 male students, 3 are are undergrad students. This just leaves 1 male grad student. - Since there's just one male grad student, their grade is $298 - 210 = 88$.

(b) - Age, Major, and Degree are the attributes that form a quasi-identifier in the table. These attributes don't directly reveal an individual's identity. However, they are intrinsic to the student, and can be cross-referenced with external databases to uniquely identify individuals. - Using the same logic, we can argue that `ID` is not a quasi-identifier because it is an identifier just for this dataset. It's not an intrinsic attribute that exists in external databases to link identities.

(c) - Yes, the selected quasi-identifier satisfies k-anonymity. - The value of k is 1. This is because the equivalence class (27, EE, PhD) appears only once.

(d) - The most effective approach for modifying the `Age` is generalizing values into broader intervals. - We can create three broad age intervals: 1. 18 - 21: groups all Bachelor students 2. 22 - 24: groups all Master students 3. 25 - 27: groups all PhD students - Applying these ranges ensures that every equivalence class contains at least 3 records. Therefore, this modification successfully achieves $k = 3$. - Another approach is to drop the `Age` column entirely, however it would lead to higher loss of data than generalization.

(e) - A dataset satisfies l-diversity if every equivalence class contains at least l distinct values for each sensitive attribute (salary in this example). - After increasing k to 3, the dataset is 3-diverse. $l = 3$. - This is because, for every single equivalence class ({18-21, CS, Bachelor}, {22-24, CS, Master}, {25-27, CS, PhD}, etc.) there are 3 distinct values in the `Salary` column (which is a sensitive field). - - - **Q2. (a)** - The unbounded sensitivity is **1**. This is because adding or removing a single cat from the dataset changes the total count of spayed cats by at most 1. - The bounded sensitivity is also **1**. This is because changing one cat's status alters the total count of spayed cats by at most 1.

(b) - The bounded sensitivity of this query is **1.25 kg**. - Since every cat's weight lies in the range {0, 10}, the maximum possible weight shift is 10 kg. - Under bounded sensitivity, the number of cats does not change. - Therefore, the mean weight will shift by $\frac{10}{8} = 1.25$kg

(c) - From (a), we know that the unbounded sensitivity $\Delta f_1 = 1$.

$$b = \frac{\Delta f_1}{\epsilon} = \frac{1}{0.25} = 4$$

- Therefore, the value of the Laplace scale parameter b using unbounded sensitivity is 4.

(d) - We know that

$$M(D) = f(D) + y$$

- We can also calculate $f_1(D) = 5$ (number of spayed cats) - Therefore, the value we will release is

$$M(D) = f_1(D) + y = 5 + 3.2 = 8.2$$

(e) - We know that

$$Pr[|Y| > t] = e^{\frac{-t}{b}}$$

- From (c), we know that $b = 4$. Therefore

$$Pr[|Y| > 2] = e^{\frac{-2}{4}} = e^{-0.5} \approx 0.6065$$

(f) - If we set $\epsilon = 0.05$, the Laplace scale parameter $b = \frac{1}{0.05} = 20$. - This results in a very large noise scale with standard deviation $\sqrt{2b^2} = 28.28$. - Because we reduced ϵ, the privacy will increase by introducing more noise. - However, the utility of this data decreases. - For example, we know that $f_1(D) = 5$. With $b = 20$, with a noise sample like $y = 22$, the value we would release is $5 + 22 = 27$ which does not make sense because the total number of cats in the dataset is only 8. - Therefore, no, the released data will not be very useful. - - - **Q3. (a)** - The most trivial solution to ensure that BuyNoSpy does not learn about $\vec{q}$ is to download the entire inventory matrix A. - Alice can then compute $\vec{q} \cdot A$ locally, without having to send $\vec{q}$ to the server.

(b) - Because Alice is not sending any data to the server, the **upload complexity is 0.** - The server sends the entire inventory matrix to Alice. Therefore, the download complexity is $m \times n = mn$ bits. - Therefore, the total

complexity is also mn bits. - Since the server sends the entire inventory matrix, each row in the matrix is likely to be the intended row. Therefore, the probability that the server correctly guesses the intended row is $\frac{1}{n}$.

(c) - Setting (i): Alice wants to obtain the details on $\frac{n}{2}$ products. - In the trivial scheme, the server sends the entire inventory matrix, and Alice does the $\vec{q} \cdot A$ computations locally. - Even if she wants to obtain the details of $n/2$ products, she needs to download the matrix just once. - Therefore, the total number of matrix elements she downloads is $m \times n = mn$ elements. - Setting (ii): $\frac{n}{2}$ customers at different locations want to obtain the details of one product each. - In this setting, because each customer is at a different location, the server must send the inventory matrix to each user. - Since there are $\frac{n}{2}$ users, the total number of matrix elements download will be

$$\frac{n}{2} \times m \times n = \frac{mn^2}{2} \text{ elements}$$

- **Setting (i) involves downloading a smaller number of matrix elements.** This is because a user can download the matrix just once and reuse it to retrieve multiple products (rows).

(d) - We know

$$\begin{aligned}
\vec{q_3} &= \vec{q_1} \oplus \vec{q_2} \oplus \vec{q} \\
\vec{s_1} &= \vec{q_1} \cdot A \\
\vec{s_2} &= \vec{q_2} \cdot A \\
\vec{s_3} &= \vec{q_3} \cdot A \\
\vec{s} &= \vec{s_1} \oplus \vec{s_2} \oplus \vec{s_3}
\end{aligned}$$

- Substituting the values of $\vec{s_1}$, $\vec{s_2}$, and $\vec{s_3}$ in the final equation, we get

$$\vec{s} = (\vec{q_1} \cdot A) \oplus (\vec{q_2} \cdot A) \oplus (\vec{q_3} \cdot A)$$

- Now we can substitute $\vec{q_3}$. Using the distributive property of XOR, we get

$$\begin{aligned}
\vec{s} &= (\vec{q_1} \cdot A) \oplus (\vec{q_2} \cdot A) \oplus ((\vec{q_1} \oplus \vec{q_2} \oplus \vec{q}) \cdot A) \\
\vec{s} &= (\vec{q_1} \cdot A) \oplus (\vec{q_2} \cdot A) \oplus (\vec{q_1} \cdot A) \oplus (\vec{q_2} \cdot A) \oplus (\vec{q} \cdot A) \\
\vec{s} &= [(\vec{q_1} \cdot A) \oplus (\vec{q_1} \cdot A)] \oplus [(\vec{q_2} \cdot A) \oplus (\vec{q_2} \cdot A)] \oplus (\vec{q} \cdot A) \\
\vec{s} &= [0] \oplus [0] \oplus (\vec{q} \cdot A) \\
\vec{s} &= \vec{q} \cdot A = A_c
\end{aligned}$$

- Therefore, at the end of the protocol, the user obtains row c of matrix A.

(e) - No, the individual servers learn no information about the desired row c. - From (d), we know that to calculate $s = A_c$, you need all $\vec{q_1}$, $\vec{q_2}$, and $\vec{q_3}$. - However, we know that the servers do not share the requests they received (non-collusion assumption). - Therefore, a single server does not have enough data to calculate s. Hence, the servers do not learn any information about the desired row.

(f) - The user uploads three $(1 \times n)$ vectors $\vec{q_1}$, $\vec{q_2}$, and $\vec{q_3}$ each of n bits. Therefore, **the upload complexity is $3n$ elements.** - Each server sends back a single $(1 \times m)$ vector of m bits. Therefore, **the download complexity is $3m$ elements.** - Therefore, **the total complexity is $3(n + m)$ elements.** - - - **Q4. (a)** - Using a Merkle tree allows verification of a specific transaction in $O(\log n)$ time and requires only $O(\log n)$ sibling nodes to be download onto the client (wallet). - On the contrary, hashing all transactions as a single concatenated string would require downloading all the transactions and re-hashing all of them again to verify. This takes $O(n)$ time and requires $O(n)$ space.

(b) - First, we must compute the hashes for the leaves. - $L_1 = H(63) = 6 + 3 = 9$ - Similarly, $L_2 = 16$, $L_3 = 11$, $L_4 = 11$, $L_5 = 7$, $L_6 = 6$, $L_7 = 10$, and $L_8 = 10$. - Next, we combine adjacent pairs using concatenation & calculate their hashes - $H_{1,1} = H(L_1||L_2) = H(916) = 9 + 1 + 6 = 16$ - Similarly, $H_{1,2} = 4$, $H_{1,3} = 13$, $H_{1,4} = 2$ - We repeat the same process again. - $H_{2,1} = 11$ - $H_{2,2} = 6$ - Finally, we calculate the hash of the root. - $H_{\text{root}} = H(116) = 8$

(c) - **Head Start:** The selfish miner gets a head start on mining subsequent blocks. Honest miners continue wasting computational power and electricity mining on the publicly known chain, while the selfish miner can move to the next blocks. - **Invalidating Honest Work:** Whenever honest miners discover a block and threaten catch up, the selfish miner releases their hidden block. The network follows the longest-chain rule; therefore the network adopts the selfish miner's longer chain and abandons the honest miner's blocks.

(d) - In Proof-of-Work, mining requires consuming physical energy and computational power. Miners are economically incentivized to concentrate all their hash power on a single long chain, because mining on a losing fork wastes electricity and other resources with zero payout. - PoS protocols resolve this by implementing slashing mechanisms. When a validator double-signs, cryptographic evidence of the double vote is submitted to the network. The protocol automatically destroys a portion of the validator's staked collateral and forcibly removes them from the validator set, making the attack economically ruinous. - - -